Projektuppgift DT071G

Konsolapplikation för Att-Göra lista

Emma Heikkinen

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Emma Heikkinen, emhe2504@student.miun.se
Utbildningsprogram: TWEUG, Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

1 Sammanfattning

Denna rapport presenterar arbetsgången för projektuppgiften i kursen DT071G. Uppgiften går ut på att skapa en applikation avsedd för konsol, desktop, webb eller mobil. I detta fall skapas en konsolapplikation i form av en Att-Göra lista som innefattar delar från kursens tidigare läromoment. Rapporten sammanfattar hur arbetet strukturerats upp, från idé till själva konstruktionen av applikationen. I slutet sammanfattas även hur resultatet blev samt vilka slutsatser som dragits under arbetets gång.

2 Innehållsförteckning

1 Sammanfattning 2
2 Innehållsförteckning 3
3 Introduktion 4
3.1 Problemmotivering 4
4 Teori 5-6
5 Metod 7
6 Konstruktion 8-10
7 Resultat 11-12
8 Slutsatser 13-14
9 Källförteckning 15

3 Introduktion

3.1 Problemmotivering

I detta projekt ska en konsolsapplikation skapas i form av en Att-göra lista. Applikationen ska innefatta delar från kursens läromoment och sammanställas till en användbar applikation. Applikationen ska fungera felfritt vid inlämning så samtliga delar i applikationen ska testas innan publicering. Applikationen ska sedan presenteras i en videopresentation och en projektrapport ska skrivas.

För att nå resultatet med en färdig applikation behövs en plan skapas och först när denna är klar kan konstruktionen starta. I planen behöver lämpliga delar tas fram som anses viktiga att ha med i en sådan här typ av applikation. När dessa sammanställts ska funktionerna utformas på ett tydligt och funktionellt sätt så att applikationen kan användas utan svårigheter att förstå den. Det ska finnas en huvudmeny med nummerval som användaren kan använda för olika funktionalitet.

Inledningsvis så ska det finnas ett val i menyn där användaren kan lägga till uppgifter i sin Att-göra lista. Användaren ska kunna svara på några frågor i konsolen och sedan ska svaren sättas ihop till en uppgift i listan. De delar som applikationen ska fråga efter är vilken uppgift användaren vill lägga till samt vilken prioritet uppgiften har från ett till tre. Uppgifterna ska visa aktuell status, detta ligger dock under en egen del. Sedan ska uppgifterna lagras i en JSON-fil.

Nästa val i menyn ska hantera status på uppgifterna. När användaren lägger till en ny uppgift i listan ska denna automatiskt markeras som ”Ej färdig” då uppgiften precis lagts till. När användaren sedan vill uppdatera statusen på en uppgift ska de kunna använda avsatt menyval för detta. När en uppgift markerats som ”Färdig” ska detta synas i listan, dels genom att statusen ändras om till ”Färdig”, men även genom att uppgiften ändrar färg.

Fortsättningsvis ska användaren kunna ta bort en uppgift från Att-göra listan. När användaren gjort detta val i menyn ska en förenklad lista med de numrerade uppgifterna visas så användaren kan välja vilken uppgift de vill radera baserat på vilket nummer uppgiften har.

Det ska även finnas en funktion i applikationen som visar statistik för hur långt användaren kommit med att bocka av sina uppgifter. Även detta ska användaren kunna göra via ett val i huvudmenyn. Lämplig statistik som ska visas är antalet uppgifter i listan, antal avklarade, antal icke avklarade, procent avklarade samt procent icke avklarade uppgifter. Även något motiverande citat ska finnas med för att göra applikationen trevligare för användaren. Ett av valen i menyn ska även stänga ned applikationen helt.

Sammanfattningsvis ska alltså en användbar och inbjudande applikation skapas. Den ska innehålla trevliga färger och figurer samt användarvänliga funktioner.

4 Teori

C#

C# är ett programmeringsspråk skapat för effektiv utveckling. Språket fungerar på samtliga plattformar och inom .NET är C# det programmeringsspråk som är mest populärt. C# är huvudsakligen ett objektorienterat programmeringsspråk, men stödjer även andra sätt att skriva och strukturera kod som till exempel funktionell programmering. I språket används bland annat klasser och metoder för att strukturera och organisera programkoden. Då C# är ”strikt typat” behöver variabler ha en angiven datatyp, tillexempel om en varibal innehåller en textsträng eller ett heltal. För att köra det program som byggts upp används kommandot dotnet run. (1)

Vid utveckling av konsolapplikationer används bland annat Console.WriteLine() för att skriva ut text och information till användaren i konsolfönstret samt Console.Write() och Console.Readline() för att ta emot svar från användaren. (1) I C# används även så kallade åtkomstmodifierare som anger vilka delar av programkoden, tillexempel variabler eller klasser, som ska kunna kommas åt av andra delar av i programkoden. (2)

.NET

.NET är en utvecklingsplattform som används för att skapa och köra program skrivna i olika programmeringsspråk, i detta fall C#. Plattformen är utarbetad för att användaren ska kunna skapa effektiva, snabba, säkra och stabila program. .NET innehåller olika delar som möjliggör utveckling, bland annat kompilering från källkod till kod som är körbar, möjlighet att använda funktioner för JSON-hantering eller användandet av ramverk som underlättar skapandet av applikationer. (3)

JSON, serialisering och deserialisering

När ett program ska lagra och hämta information finns det processer som kan användas för att hantera data. I detta projekt hämtas data från en användare via konsolen, vilket sedan ska sparas i en JSON-fil. För att möjliggöra detta används två processer:

Serialisering: Informationen i programmet, till exempel objekt och deras egenskaper, omvandlas till JSON-format för att kunna lagras.

Deserialisering: Den data som serialiserats till JSON-format omvandlas tillbaka till det format som programmet behöver för att kunna läsa och använda det. (4)

SOLID-principer

Vi programmering är det önskvärt att koden skapas på ett hållbart vis och så den är lätt att underhålla. Allteftersom koden byggs på med flera delar är SOLID-principerna användbara för att en ren och kvalitativ kod, vilket i längden också bidrar till en säker kod. Det finns fem SOLID-principer som är bra att följa vid programmering. Första principen anger att klasser endast ska ha ett ansvar. Andra principen anger att det ska vara möjligt att addera ytterligare funktioner till ett program utan att behöva ändra den befintliga koden. Tredje principen innebär att underklasserna ska kunna ersätta sina överklasser utan påverkan på programmets funktion. Fjärde principen fastställer att klasser endast ska behöva implementera metoder de faktiskt har användning för. Sista och femte principen innebär att beroenden mellan konkreta klasser bör undvikas, utan att klasser i stället bör vara beroende av abstrakta klasser eller gränssnitt. (5)

Teckenkodning i konsolen

I programmet har följande implementerats, se bilaga 1.1.

```
Console.OutputEncoding = Encoding.UTF8;
Console.InputEncoding = Encoding.Unicode;
```
(1.1)

Utdatakodning (UTF-8) används för att de tecken som skrivs ut i konsolen, till exempel Å, Ä, och Ö, ska visas korrekt. (6)

Indatakodning används för motsatsen. Tecken som användaren skriver in i konsolen ska översättas till tecken som programmet kan tolka. (7)

5 Metod

Denna uppgift kommer lösas i flera steg. Word kommer användas för att skapa flera delar. En projektplan kommer skapas för att sammanfatta de delar som ska finnas med i applikationen och tydliggöra utvecklingen. Planen kommer bli ett stöd under arbetets gång. Även ett klassdiagram kommer skapas för att visa programmets klasser och metoder. Likaså kommer en rapport att skrivas.

Därefter kommer ett nytt projekt startas upp. För uppgiften kommer Visual Studio Code och Node.js command prompt att användas. Applikationen kommer att skapas med programmeringsspråket C# och med ramverket .NET. Därefter kommer filstrukturen skapas. Projektet kommer även publiceras till GitHub för versionshantering.

Filerna som ska användas i applikationen kommer delas upp så att varje fil döps utefter de specifika delar den innehåller. De klasser och metoder som varje fil innehåller kommer struktureras enligt delen i SOLID-principerna som förtydligar att varje klass eller modul endast ska ha ett jobb eller ändamål. (5) Inledningsvis kommer filerna Program.cs, ToDo.cs, ListFunctions.cs och Statistics.cs att skapas.

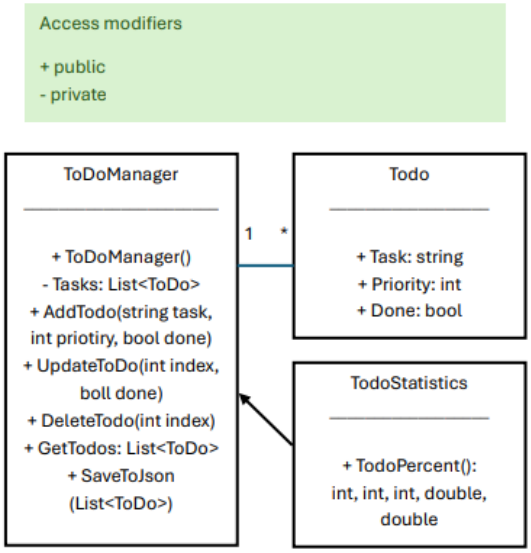
Efter detta kommer innehållet i respektive fil att utformas likt följande:
Program.cs: kommer innehålla applikationens start. Här ska applikationens huvudmeny skapas med 5 olika val, en för varje funktion som applikationen innehåller. En switch-sats kommer användas för användarinteraktionerna, så när användarens val tas emot ska svaret läsas in och resultera i olika funktioner.
ToDo.cs: kommer skapas för att innehålla en uppgifts struktur. Strukturen för en uppgift kommer vara uppgift, prioritet och status. Get/set kommer användas för att läsa och tilldela värden till egenskaperna.
ListFunctions.cs: kommer skapas för att sköta Att-göra listans funktioner. Klassen ToDoManager kommer skapas för att innehålla de metoder som sköter respektive funktion. Funktionerna som ska finnas med är skapa, ändra och ta bort en uppgift samt möjlighet att läsa in alla uppgifter. För att addera och ta bort uppgifter kommer .Add() samt .RemoveAt() att användas.
Statistics.cs: kommer skapas med en klass och en metod för att beräkna Att-göra listans statistik, det vill säga hur användaren har lyckats med att bocka av sina uppgifter. Antal genomförda och ej genomförda uppgifter kommer räknas genom en foreach-loop, medan procenten avklarade och icke avklarade uppgifter kommer beräknas genom en matematisk formel för procenträkning.

Applikationen kommer även innehålla en metod för JSON-hantering. Här kommer serialisering och deserialisering användas för att omvandla data.

När applikationen är klar och allt fungerar som det ska kommer en videopresentation att spelas in med OBS studios för att visa applikationens funktioner samt delar ur programkoden. Sedan kommer projektarbetet att lämnas in för rättning.

6 Konstruktion

Innan uppbyggandet av själva applikationen startade skapades en planering. I planeringen, som skrevs i Word, sammanfattades kort vilka delar som skulle finnas med i applikationen och vilka funktioner som skulle vara användbara. Under arbetets gång uppdaterades planeringen även med ett klassdiagram över de klasser som enligt plan skulle användas i applikationen, se bilaga 1.2. När planen för applikationen var klar startade arbetet med konstruktionen. Ett nytt projekt skapades i Visual Studio Code och ett repository i GitHub initierades. Sedan skapades program.cs, ListFunctions.cs och ToDo.cs.



(1.2)

I ToDo.cs skapades klassen ToDo som innehåller uppbyggnaden av en uppgift i Att-Göra listan, det vill säga vad som ska göras, prioritet och status. Se bilaga 1.3.

```
//Public class ToDo
12 references
public class ToDo
{
    4 references
    public string? Task { get; set; }

    3 references
    public int? Priority { get; set; }

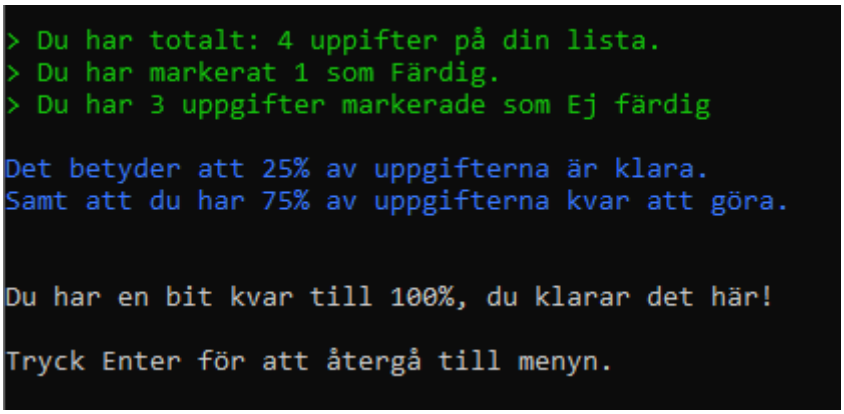
    4 references
    public bool Done { get; set; }
}
```

(1.3)

Sedan inleddes arbetet med program.cs, alltså programmets startpunkt. Filen innehåller applikationens startsida som är konstruerad att skriva ut Att-Göra listan tillsammans med huvudmenyn som användaren ska använda. Menyn som användaren kan använda har skapats med 5 numrerade rader, där varje siffra innehåller en funktion likt följande:

1. Lägg till uppgift
2. Uppdatera status
3. Ta bort uppgift
4. Se din list-statistik
4. Avsluta

- Val ”1” är konstruerad så att användaren får skriva in vilken uppgift de vill addera till Att-Göra listan samt vilken prioritet (1–3) uppgiften har. Sedan informeras användaren även om att uppgiften inledningsvis markeras om ”Ej färdig”.
- Val ”2” är konstruerad så att användaren får frågan om vilken uppgift de vill ändra status på. Användaren får ange nummer på den uppgift vars status ska uppdateras och sedan svara ja eller nej på frågan om uppgiften är utförd. Om användaren svarar jag ändras statusen på uppgiften till ”Färdig” samtidigt som den får en annan färg i Att-Göra listan. Färgändringen lades till i applikationen för att ”Färdiga” och ”Ej färdiga” uppgifter tydligare ska kunna särskiljas när användaren överblickar Att-Göra listan.
- Val ”3” är konstruerad så att användaren får ta bort en uppgift från listan. Skärmen rensas och därefter presenteras en rensad version av Att-Göra listan med endast numrerade uppgifter, men då utan prioritet och status. Användaren får här ange nummer på uppgiften de vill ta bort.
- Val ”4” är konstruerad så att användaren får se sin list-statistik, alltså hur det går för användaren med att bocka av uppgifter på sin lista. Statistiken skrivs ut enligt ordningen i bilaga 1.4.
- Val ”5” är konstruerad att stänga ner hela applikationen.



(1.4)

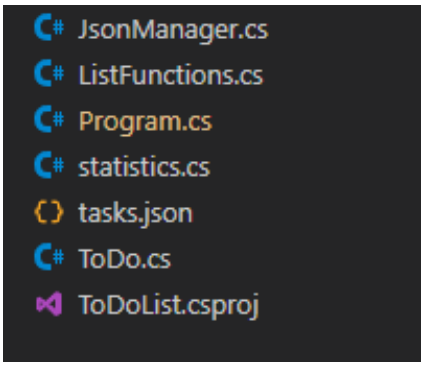
I uppbyggandet av applikationen har vissa delar adderats för att skapa en inbjudande känsla i applikationen. Dels så används figurer, men även en uppmuntrande mening. Meningen är konstruerad att skrivas ut olika beroende på hur många uppgifter användaren klarat av. Olika meningar skrivs ut för under 75%, över 75% och 100% avklarade uppgifter.

ListFunctions.cs är skapad för att innehålla alla listfunktioner som nämns ovan. Klassen ToDoManager innehåller metoder för att addera en uppgift, uppdatera en uppgift, radera en uppgift samt en metod för att hämta alla uppgifter. I början av uppbyggandet av applikationen innehöll denna fil även en metod för JSON-hantering. Detta korrigerades dock under arbetets gång och därefter lades filen

JsonManager.cs till. JsonManager.cs är således skapad för att JSON-hantering. Metoderna i denna fil serialiserar listan till JSON-format och deserialiserar listan från JSON-format tillbaka till formatet programmet använder.

Statistics.cs är tillverkad för att sköta uträkningarna för applikationens statistik. Metoden i denna klass är konstruerad att göra 5 olika räkne moment likt följande:
- Räkna totalt antal uppgifter i Att-Göra listan genom list.Count().
- Räkna antalet ”Färdiga” uppgifter, alltså uppgifter där Done = true.
- Räkna antalet ”Ej färdiga” uppgifter, alltså uppgifter där Done = false.
- Räkna ut hur stor procent av alla uppgifter som är avklarade genom att ta antalet avklarade uppgifter, dividera dem med det totala antalet uppgifter och sedan multiplicera detta med 100.
- Räkna ut hur stor procent av alla uppgifter som inte är avklarade genom att ta antalet ej avklarade uppgifter, dividera dem med det totala antalet uppgifter och sedan multiplicera detta med 100.

Uppbyggnaden av applikationen har sammanfattningsvis ställts inför vissa ändringar då programkodens struktur kunnat blir mer förenlig med principen om att klasser ska ha egna ansvar. Strukturen för filer och programkod samt förändringar som skett under utvecklingen har presenterats i skrift, men en förtydligande bild av den slutgiltiga filstrukturen visas i bilaga 1.5.



(1.5)

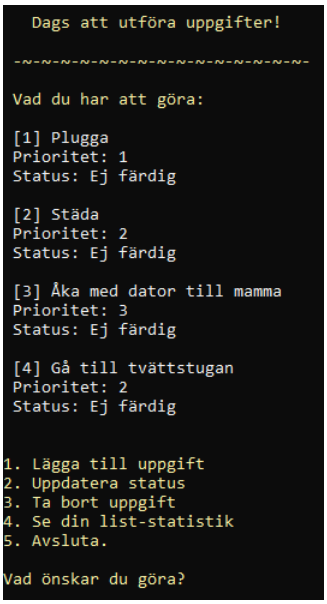
Därefter avslutades projektarbetet med att skapa en videoinspelning. Videoinspelningen skapades i OBS Studios och spelades in med syfte att visa de centrala delarna i applikationen samt delar ut programkoden som ansågs relevanta.

Projektets rapport har skapats enligt MittUniversitetets rapportmall och utifrån instruktionen om rubriksstrukturen som finns i Canvas.

7 Resultat

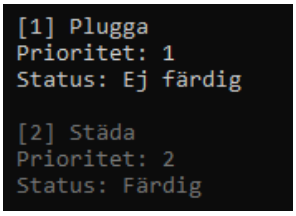
Projektarbetet, som utförts i flera steg, har resulterat i en enkel projektplanering, ett klassdiagram som visar den inledande planen för klasstrukturen (se bilaga 1.2), en färdig applikation, en videoinspelning som visar applikationen och delar ut programkoden samt en projektrapport.

Konsolapplikationen är en färdig Att-Göra lista, där startsidan visar alla uppgifter som finns i listan samt en huvudmeny. Se bilaga 1.6.



(1.6)

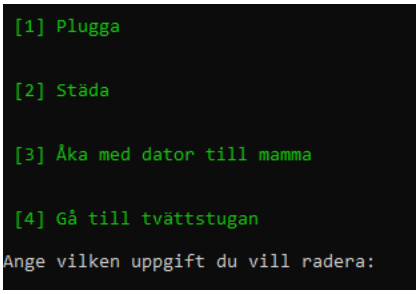
Användaren kan använda de numrerade alternativen i menyn för att nyttja applikationens funktioner. De olika nummervalen resulterar i:
1 – Applikationen ber användaren ange vilken uppgift de vill lägga till i listan samt vilken prioritet uppgiften ska ha från ett till tre. Uppgiftens status anges automatiskt till ”Ej färdig”. En numrerad uppgift läggs därefter till i listan och innehåller det användaren angivit.
2 – Applikationen ber användaren ange nummer på den uppgift vars status de vill ändra samt ange om uppgiften är utförd. När användaren anger ”ja” på frågan om uppgiften är utförd förändras statusen på uppgiften till färdig och hela uppgiften ändrar färg. Se bilaga 1.7.



(1.7)

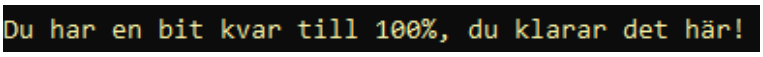
3 – Applikationen rensar konsolen och skriver sedan ut en förenklad Att-Göra lista, där endast uppgiften med tillhörande nummer visas. Se bilaga

1.8. Därefter frågar applikationen vilken uppgift användaren vill ta bort. När användaren anger uppgiftens nummer försvinner uppgiften från listan.

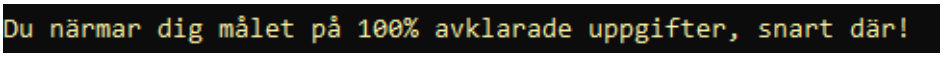


(1.8)

4 – Applikationen visar statistik över hur det går för användaren med att bocka av uppgifter. Användaren förses med det totala antalet uppgifter i listan, antalet avklarade uppgifter, antalet icke avklarade uppgifter, procent avklarade uppgifter samt procent icke avklarade uppgifter. Applikationen ger även användaren en uppmuntrande mening beroende på hur många procent av uppgifterna i listan denne har bockat av:
- Mindre än 75% avklarade uppgifter, se bilaga 1.9.
- Mer än 75% avklarade uppgifter, se bilaga 1.10.
- 100% avklarade uppgifter, se bilaga 1.11.



(1.9)



(1.10)



(1.11)

5 – Applikationen avslutas och stängs ner.

Applikationen är testad flera gånger och vid varje test har applikationens funktioner kontrollerats. Resultatet vid alla testar har visat att funktionerna fungerar utan problem eller uppstådda fel.

Applikationen är skapad för att leverera resultat som följer projektrapportens problemformulering. Därmed skiftar applikationens utskrifter i olika färger och likaså finns två olika figurer med i applikationen.

8 Slutsatser

Nu när applikationen är färdig och dess funktionalitet har fungerat ändamålsenligt i samtliga tester, ger återblicken över projektet en känsla av att arbetet gått mestadels bra. Planen för applikationens funktionalitet har efterföljts och varje val i huvudmenyn ger nu användaren tillgång till den specifika funktion i applikationen det var avsett för.

Tanken med att skapa denna typ av applikation var att använda lärdomar från de tidigare momenten i kursen och sedan utveckla dessa vidare i en ny form av konsolapplikation. Eftersom vi tidigare hade arbetat med objektorienterad programmering, JSON-hantering och generiska listor som lagrar element var tanken att tillämpa dessa kunskaper och sedan vidareutveckla dem i projektuppgiften. Därför har fokus, i arbetet med applikationen, legat på att försöka skapa en tydlig struktur i programkoden och då speciellt med avsikten att följa SOLID-principen om enskilda ansvar. Utöver detta har även applikationen försetts med olika delar som tillsammans ger en något mer utvecklad Att-Göra lista, snarare än en lista som endast erbjuder grundläggande delar. Ett exempel på en funktion som lagts till i listan för att utveckla innehållet är statistikberäkningen.

Något som kunde har gjorts bättre under projektarbetet är att skapa en mer utvecklad planering, men också att följa planeringen lite mer noggrant. I stället för att så ivrigt sätta i gång med arbetet att utveckla applikationen kunde ett mer utförligt förarbete gjort att vissa delar inte hade behövt korrigeras under arbetets gång. Ett exempel på detta är upplägget för JSON-hanteringen i koden. Under tidigare föreläsningar har SOLID-principen om enskilda ansvar varit i fokus, men trots detta missades detta i inledningen av utvecklingen. Som konsekvens av detta fick filstrukturen korrigeras när arbetet med applikationen var på sluttampen och en ny fil med en ny klass och ny metod fick skapas för JSON-hantering. Det är inte ett så omfattande misstag att det får allvarliga konsekvenser för projektet eller applikationen, men ett optimalt arbetsflöde innefattar inte korrigeringar i sista minuten.

Beträffande vidareutveckling av applikationen så finns det flera delar som kan arbetas med för att skapa en mer avancerad applikation. I första hand kan programkoden med säkerhet utvecklas ytterligare för att bli mer stabil och ren samt för att följa SOLID-principerna i högre grad. Utöver detta så kan applikationen utvecklas med ytterligare funktioner för att bli mer komplex och avancerad. Ett exempel på något som kunde gjort applikationen mer utvecklad är att lagra Att-Göra uppgifterna i en databas snarare än en JSON-fil. Andra möjliga förbättringsområden för applikationen är att använda ramverk i utvecklingsarbetet samt att vidareutveckla den till en applikation för mobil, desktop eller webb. Det ska dock tilläggas att avsikten med projektarbetet aldrig har varit att utveckla en applikation som är så komplex och avancerad som den potentiellt kunde vara. Tanken med detta val av applikation har hela tiden varit att föra samman kunskaper från tidigare moment i kursen och sedan utveckla applikationen något ytterligare för att skapa en applikation passande för ett

projektarbete på grundläggande nivå.

Hela kursen i C#.NET har bidragit med grundläggande kunskaper i programmeringsspråket C#, men projektarbetet har därefter bidragit med möjligheten att sammansätta kunskaperna och använda dessa i en mer utvecklad applikation. Flera nya kunskaper har inhämtats om C# som programmeringsspråk, däribland hur ett strikt typat och objektorienterat programmeringsspråk ger möjlighet till att skapa en ren, lätthanterlig och säker kod. Med det sagt så finns det fortfarande mycket kvar att lära och utforska inom området för att bli ännu bättre.

9 Källförteckning

[1] Microsoft Ignite, ”En rundtur i C#-språket” https://learn.microsoft.com/sv-se/dotnet/csharp/tour-of-csharp/overview Publicerad 2026-02-24. Hämtad 2026-10-30.

[2] Microsoft Ignite, ”Åtkomstmodifierare (C#-programmeringsguide)” https://learn.microsoft.com/sv-se/dotnet/csharp/programming-guide/classes-and-structs/access-modifiers Publicerad 2025-12-17. Hämtad 2026-10-30.

[3] Microsoft Ignite, ”Introduktion till .NET” https://learn.microsoft.com/sv-se/dotnet/core/introduction Publicerad 2026-03-14. Hämtad 2026-10-30.

[4] Microsoft Ignite, ”JSON-serialisering och deserialisering i .NET - översikt” https://learn.microsoft.com/sv-se/dotnet/standard/serialization/system-text-json/overview Publicerad 2025-12-05. Hämtad 2026-10-30.

[5] STRATO, ”Vad är SOLID-principerna” https://www.strato.se/guide/webbplatser/webbutveckling/solid-principerna/ Publicerad 2025-01-14. Hämtad 2026-10-31.

[6] Microsoft Ignite, ”Console.OutputEncoding Property” https://learn.microsoft.com/en-us/dotnet/api/system.console.outputencoding?view=net-10.0 Publicerad ”u.å.”. Hämtad 2026-10-31.

[7] Microsoft Ignite, ”Console.InputEncoding Property” https://learn.microsoft.com/en-us/dotnet/api/system.console.inputencoding?view=net-10.0 Publicerad ”u.å.”. Hämtad 2026-10-31.